Fundamentals of Natural Language Processing

Chapter 2 — Text Representation and Evaluation

Aymen Ben Brik

Chapter 2

Text Representation and Evaluation

A computer cannot operate on raw text. Before any model can "read" a sentence, the sentence must be *tokenized* (cut into discrete units), *numericalized* (converted into integer indices), and *vectorized* (embedded in a vector space). This chapter develops the chain rigorously: tokenization granularities, sparse and dense vector-space models, semantic similarity, and the evaluation metrics by which we compare NLP systems.

2.1 Tokenization fundamentals

2.1.1 Motivation

Tokenization is the unsung hero of every NLP pipeline. A poor tokenizer silently undermines the most sophisticated downstream model: out-of-vocabulary words become unknown tokens, morphological variants are treated as unrelated, multi-word expressions are fragmented. Conversely, a tokenizer aligned with the data distribution can shrink vocabulary size, accelerate training, and improve generalisation. Modern Large Language Models all rely on a single technical choice — *sub-word tokenization* — whose strengths and limitations every practitioner should understand.

2.1.2 Approach by problem: how many tokens are in this sentence?

Problem. Consider the sentence

> *"I've been to Sfax-University; it's amazing!"*

Count how many "tokens" it contains under each of the following naive policies:

1. split on whitespace;
2. split on whitespace and punctuation;
3. split into individual characters.

Solution sketch.

1. Whitespace only: {`I've`, `been`, `to`, `Sfax-University;`, `it's`, `amazing!`} — 6 tokens. Punctuation glued to words; "I've" is a single token.

2. Whitespace and punctuation: {`I`, `'ve`, `been`, `to`, `Sfax`, `-`, `University`, `;`, `it`, `'s`, `amazing`, `!`} — 12 tokens. The contraction *I've* is split, but so is the legitimate compound *Sfax-University*.

3. Characters: 35 tokens. Punctuation, capitalisation and spaces are all preserved, but every word is fragmented; the model would have to relearn "what is a word" from data.

Each policy entails a trade-off between vocabulary size and information preserved. There is no universally correct answer; the choice is engineering, guided by the downstream task and the language being modelled.

2.1.3 Definition and the tokenization pipeline

Definition 2.1.1 (Tokenization)**. Tokenization** is the process of converting a raw text string into an ordered sequence of discrete units called **tokens**, drawn from a finite inventory called the **vocabulary** V.

Definition 2.1.2 (Numericalization)**. Numericalization** is the bijective mapping $V \leftrightarrow \{0, 1, \ldots, |V| - 1\}$ that assigns each token an integer identifier.

The full pipeline is: *raw text* $\rightarrow$ tokens $\rightarrow$ token IDs $\rightarrow$ tensor inputs to a model. Every NLP system is built on top of these three trivially simple but practically critical steps.

2.1.4 Three levels of granularity

Level	Vocabulary size	Sequence length	OOV behaviour
Word	Very large (10^5–10^6)	Short	Unknown words become `[UNK]`
Character	Tiny (~ 100)	Very long	No OOV (any character is in V)
Sub-word	Moderate (10^4–10^5)	Moderate	Robust: any word splits into known pieces

Word tokenization. The intuitive default. Tokens are entire words separated by whitespace and punctuation rules. Strengths: each token carries lexical meaning, and downstream models converge faster on a small annotated dataset. Weaknesses: vocabulary explodes for morphologically rich languages (*Arabic, Turkish, Finnish*); typos and rare words become `[UNK]`; cannot generalise across morphological variants (*run / runs / running* are three unrelated tokens).

Character tokenization. Each character is a token. Strengths: vocabulary is bounded and stable across languages; no out-of-vocabulary problem (any unseen word becomes a sequence of known characters). Weaknesses: sequences become 5–10$\times$ longer, increasing both memory and compute; the model must learn from scratch which character sequences form meaningful units.

Sub-word tokenization. The dominant choice in 2020s NLP. Vocabulary contains *frequent words as wholes* and *rare words split into meaningful pieces*. The split is learnt from data using algorithms such as Byte-Pair Encoding (BPE), WordPiece (BERT) and SentencePiece (XLM-R, T5). Sub-word tokenization combines the strengths of word-level (semantic units for common words) and character-level (no OOV) approaches.

2.1.5 Pre-processing operations

Tokenization is often combined with ad-hoc transformations that modify or normalise the input string before splitting.

- **Lowercasing**: "Apple" and "apple" become equivalent. Useful for spelling-insensitive tasks but destroys signal in NER (a capitalised *Apple* is more likely the company).
- **Punctuation removal**: simplifies vocabulary but loses sentence boundaries.
- **Stop-word removal**: discard high-frequency words (*the, of, in*) that carry little task-specific information. Useful for keyword search; harmful for syntax-sensitive tasks.
- **Stemming**: chop morphological suffixes by hand-coded rules (Porter, Snowball). *running* $\to$ *run*. Fast but linguistically crude.
- **Lemmatization**: map a word to its dictionary form using a morphological analyser. *better* $\to$ *good*. Slower but linguistically principled.

Remark 2.1.1. Modern large language models perform little or no manual pre-processing: they are trained on raw text with sub-word tokenization, and the model itself learns whatever normalisations it needs. Heavy pre-processing is typical of statistical-era pipelines and remains useful for small-data, classical-ML systems.

2.1.6 Byte-Pair Encoding (BPE) in detail

BPE is the simplest sub-word algorithm and the workhorse of GPT-2, GPT-3, GPT-4, RoBERTa, and many others. It iteratively merges the most frequent pair of adjacent symbols.

Algorithm.

1. Start with a vocabulary that contains every individual character of the corpus.
2. Treat each word as a sequence of characters with an end-of-word marker.
3. Count the frequency of every adjacent symbol pair across the corpus.
4. Merge the most frequent pair into a new symbol; add it to the vocabulary.
5. Repeat steps 3–4 until a target vocabulary size is reached.

Example 2.1.1 (A miniature BPE run)**.** Suppose the corpus consists of the words `low`, `lower`, `newest`, `widest`, with frequencies $5, 2, 6, 3$. The initial vocabulary is {`l, o, w, e, r, n, s, t, i, d, </w>`}. Counting pairs:

$$\texttt{e s} = 6 + 3 = 9, \quad \texttt{s t} = 9, \quad \texttt{l o} = 7, \quad \texttt{o w} = 7, \ldots$$

The pair `e s` (9 occurrences) is merged first into a new symbol `es`. Recount, merge again — now `es t` (9 occurrences) into `est`. Continue: `lo`, `low`, `ne`, `new`, … After enough merges, frequent words like `low</w>` become single tokens, while rare words remain decomposed. A new word like `lowest` is then tokenized as `low + est`, even though it never appeared in training.

Why BPE works. The algorithm provides a natural rate–distortion trade-off. As vocabulary grows, the average sequence length shrinks, but the "unit cost" per token increases (each token carries more information). Empirically, vocabularies of 30k–100k sub-words give an excellent trade-off across many languages.

2.1.7 Multilingual and dialectal challenges

Tokenization choices that are adequate for English may fail for other languages.

- **Whitespace-free scripts.** Chinese, Japanese, Thai write without spaces. Word tokenization requires segmentation algorithms (e.g. Jieba, MeCab).
- **Morphologically rich languages.** Arabic and Turkish concatenate prefixes and suffixes onto a stem, generating very large surface-form inventories. Sub-word tokenization is essential.
- **Code-switching and dialects.** Tunisian Arabic frequently mixes Modern Standard Arabic, French, and Latin-script transliterations ("arabizi"). A general-purpose multilingual tokenizer (e.g. XLM-R's SentencePiece) is recommended over single-language tokenizers.
- **Right-to-left scripts.** Display order and logical order can differ; numerical IDs operate on logical order, but evaluation tools must be aware of the visual rendering.

2.1.8 Worked examples

Example 1 (Granularity choice — Easy). You are training a classifier on 1000 short SMS messages, mostly in English with frequent typos. Which tokenization granularity is most appropriate, and why?

Solution. Sub-word tokenization. Word-level would suffer from OOV (typos produce never-seen surface forms); character-level would lengthen sequences and dilute the signal in a small dataset. Sub-word splits typos into recognised pieces (`hapy` → `ha + py`) without exploding the vocabulary.

Example 2 (BPE iteration — Medium). Apply two iterations of BPE to the toy corpus {`aaab`, `aabb`, `abab`} (each of frequency 1). Show the resulting vocabulary after each merge.

Solution. Initial vocabulary: {`a`, `b`}.

Iteration 1: count pairs — `a a` appears in `aaab` (twice) and `aabb` (once) and `abab` (zero) = 3; `a b` appears in `aaab` (once) and `aabb` (once) and `abab` (twice) = 4; `b a` appears once (in `abab`); `b b` once. Most frequent: `a b` (4). Merge into `ab`. Vocabulary: {`a`, `b`, `ab`}. Re-tokenization: `aaab` → `a a ab`; `aabb` → `a ab b`; `abab` → `ab ab`.

Iteration 2: count new pairs — `a a` (in `aaab`, once) = 1; `a ab` (in `aaab` and `aabb`) = 2; `ab b` (in `aabb`) = 1; `ab ab` (in `abab`) = 1. Most frequent: `a ab` (2). Merge into `aab`. Vocabulary: {`a`, `b`, `ab`, `aab`}.

Example 3 (Tokenizer mismatch — Hard). A pretrained English BERT tokenizer applied to Tunisian Arabic in Latin script ("arabizi") splits the word `n7ebek` ("I love you") as `[n, ##7, ##e, ##bek]`. Discuss the consequences for a downstream classifier and propose two remediation strategies.

Solution.

- **Consequences.** The numeric digit 7 (which substitutes for the Arabic letter ʿ) is treated as foreign; the model sees an unfamiliar pattern with no semantic anchoring. The classifier's representation of the word will be near-random, and its predictions for sentences containing arabizi will be poor.
- **Remediations.**
  1. Use a multilingual tokenizer (e.g. `xlm-roberta-base`'s SentencePiece) trained on much broader data including Latin-script Arabic.
  2. Train a domain-specific tokenizer on a Tunisian-Arabic corpus before fine-tuning the model. The merges will then capture frequent dialectal sub-words natively.

2.1.9 Python snippet: comparing tokenizers

```python
# pip install nltk transformers
import nltk
from transformers import AutoTokenizer

text = "I've␣been␣to␣Sfax-University;␣it's␣amazing!"

# Word-level (NLTK)
nltk.download('punkt_tab', quiet=True)
print("Word␣tokens␣(NLTK):", nltk.word_tokenize(text))

# Sub-word (BERT WordPiece)
bert = AutoTokenizer.from_pretrained("bert-base-uncased")
print("BERT␣sub-word␣tokens:", bert.tokenize(text))

# Sub-word (GPT-2 byte-level BPE)
gpt2 = AutoTokenizer.from_pretrained("gpt2")
print("GPT-2␣BPE␣tokens:␣␣␣␣", gpt2.tokenize(text))
```

The same input produces different tokenisations under different algorithms. Reading the tokenized output is the first debugging step when a model behaves unexpectedly.

2.1.10 Exercises

1. For each scenario, recommend a tokenization granularity (word / character / sub-word) and justify briefly:
   (a) Building a spelling corrector for French SMS;
   (b) Training a sentiment classifier on 50 million Web reviews;
   (c) Building a name-entity recognizer for German legal documents.
2. Explain in one paragraph why character-level tokenization, despite its $|V| \approx 100$, has *not* replaced sub-word tokenization in modern LLMs.
3. Tokenize the sentence "*The CEOs' meeting was rescheduled.*" with two different policies (whitespace+punctuation, and a sub-word tokenizer of your choice). Comment on the differences for the words *CEOs'* and *rescheduled*.

4. Carry out three iterations of BPE on the toy corpus $\{\texttt{xay}, \texttt{xay}, \texttt{yax}, \texttt{xy}\}$ where each word has frequency 1. List the new vocabulary at each step.

5. (Synthesis.) For a multilingual Tunisian dialect dataset (mixing Arabic script, Latin-script arabizi, and French), discuss the trade-offs between (i) using a single multilingual sub-word tokenizer (e.g. XLM-R), (ii) training a domain-specific tokenizer on the dataset, and (iii) using language-specific tokenizers for each script and merging the outputs.

2.2 Sparse representations: Bag-of-Words and TF–IDF

2.2.1 Motivation

After tokenization, a text is a sequence of integer IDs. To feed such a sequence to a classical machine-learning algorithm — logistic regression, k-nearest-neighbours, support-vector machine — we need a *fixed-size vector* representation that does not depend on document length. The simplest such representation is the **Bag-of-Words** (BoW), which counts how often each vocabulary item appears. BoW and its weighted refinement **TF–IDF** dominated information retrieval and text classification for two decades; they remain strong baselines, fast to compute, easy to debug, and competitive on small or domain-specific data sets.

2.2.2 Approach by problem: are these two reviews similar?

Problem. Compare the following two hotel reviews:

- d_1: "*The room was clean. The room was quiet.*"
- d_2: "*The breakfast was great. The room was clean.*"

Propose a numerical representation for each that lets us measure their similarity, ignoring word order.

Solution sketch. Build a vocabulary by collecting all distinct words: $V = \{$the, room, was, clean, q (size 7). For each document, count how often each vocabulary word appears:

$$\mathbf{x}_{d_1} = (2, 2, 2, 1, 1, 0, 0)^\top, \quad \mathbf{x}_{d_2} = (2, 1, 2, 1, 0, 1, 1)^\top.$$

The two vectors share many components (e.g. both contain "the", "room", "was", "clean") and differ on others (*quiet* only in d_1, *breakfast / great* only in d_2). Their dot product gives a coarse measure of similarity. This embarrassingly simple recipe is the starting point of all classical NLP.

2.2.3 Vector space models

Definition 2.2.1 (Vector space model)**.** A **vector space model** (VSM) of a corpus is an embedding $\phi : \mathcal{T} \to \mathbb{R}^d$, where $\mathcal{T}$ is the set of textual objects (words or documents) and d is the embedding dimension. Documents are represented as vectors so that geometric operations (dot product, cosine similarity, distance) translate into linguistic operations (similarity, retrieval, clustering).

2.2.4 Bag-of-Words

Definition 2.2.2 (Bag-of-Words representation)**.** Fix a vocabulary $V = \{w_1, w_2, \ldots, w_{|V|}\}$. The **Bag-of-Words** (BoW) representation of a document d is the vector

$$\phi_{\text{BoW}}(d) = \big(\text{tf}(w_1, d),\ \text{tf}(w_2, d),\ \ldots,\ \text{tf}(w_{|V|}, d)\big) \in \mathbb{R}^{|V|},$$

where $\text{tf}(w, d)$ is the *term frequency* (raw count) of word w in d. The representation is named "bag" because the order of words is discarded: the document is treated as a multiset.

Alternative weightings of tf**.**

- **Boolean**: $\text{tf}(w, d) \in \{0, 1\}$, indicating presence.
- **Raw count**: $\text{tf}(w, d) =$ number of occurrences.
- **Log-normalized**: $\text{tf}(w, d) = \log(1 + \#(w, d))$, mitigating the bias toward long documents.
- **Length-normalized**: divide raw counts by document length so that all documents have ℓ_1-norm 1.

2.2.5 The document–term matrix

Definition 2.2.3 (Document–term matrix)**.** Given a corpus $\{d_1, \ldots, d_N\}$ and a vocabulary V, the **document–term matrix** is the matrix $X \in \mathbb{R}^{N \times |V|}$ whose entry $X_{ij} = \text{tf}(w_j, d_i)$.

In practice $|V|$ is in the tens or hundreds of thousands, while only a few dozen distinct words appear in each document. The matrix is therefore extremely *sparse*: a vast majority of entries are zero. Efficient storage formats (compressed sparse row, CSR; compressed sparse column, CSC) preserve memory by storing only non-zero entries.

2.2.6 Limitations of Bag-of-Words

- **No word order**: the documents "*Tunisia beat Egypt*" and "*Egypt beat Tunisia*" yield identical BoW vectors despite their opposite meanings.
- **Equal weight to all words**: a function word like "*the*" contributes the same magnitude as a domain-specific term like "*Paracetamol*", even though the latter is far more discriminative.
- **No semantic similarity**: "*cat*" and "*kitten*" are orthogonal vectors in $\mathbb{R}^{|V|}$ — as far apart as "*cat*" and "*democracy*".
- **Curse of dimensionality**: $|V| \approx 50{,}000$ produces vectors that are computationally cheap (sparse) but statistically noisy — many features contribute marginally to any specific task.

The first two limitations are tackled in this section by TF–IDF; the third requires dense embeddings (Section **??**). The first — the loss of order — is fundamentally unfixable in a bag-of-words framework and is a key driver of the move to recurrent and Transformer models.

2.2.7 TF–IDF

The intuition is to *down-weight* words that appear in many documents (because they discriminate poorly between documents) and *up-weight* words that appear only in a few (because their presence is informative).

Definition 2.2.4 (Inverse document frequency)**.** Given a corpus of N documents, the **document frequency** of a word w is the number of documents that contain w at least once,

$$\mathrm{df}(w) = \big|\{d \in \text{corpus} : w \in d\}\big|.$$

The **inverse document frequency** is

$$\mathrm{idf}(w) = \log \frac{N}{\mathrm{df}(w)}.$$

Common variants smooth the formula to avoid division by zero and dampen extreme values:

$$\mathrm{idf}_{\text{smooth}}(w) = \log\left(\frac{N+1}{\mathrm{df}(w)+1}\right) + 1.$$

Definition 2.2.5 (TF–IDF)**.** The **TF–IDF** weight of word w in document d is the product

$$\mathrm{tfidf}(w, d) = \mathrm{tf}(w, d) \times \mathrm{idf}(w).$$

Replacing each entry of the document–term matrix by the corresponding TF–IDF gives the TF–IDF document–term matrix.

Intuition.

- Words appearing in nearly every document have $\mathrm{df}(w) \approx N$, hence $\mathrm{idf}(w) \approx \log 1 = 0$. They are zeroed out — the same effect as stop-word removal, but data-driven.
- Words appearing in few documents have $\mathrm{df}(w) \ll N$, hence large $\mathrm{idf}(w)$. They retain (and amplify) their term-frequency contribution.
- Within a single document, TF–IDF still scales with raw term frequency, preserving emphasis on repeated key terms.

Property 2.2.1 (Boundedness of idf). For a corpus of N documents, $0 \le \mathrm{idf}(w) \le \log N$ when idf uses the unsmoothed formula. The maximum is reached when w appears in exactly one document, the minimum when w appears in all of them.

2.2.8 TF–IDF as a probabilistic measure

Remark 2.2.1 (Information-theoretic flavour). $\mathrm{idf}(w) = -\log \frac{\mathrm{df}(w)}{N}$ is the negative log-probability that a randomly chosen document contains the word w. A rare word "surprises" us when it appears, contributing many bits of information; a common word contributes few. This is a discrete analogue of pointwise mutual information, and the connection has been formalised: TF–IDF approximates a maximum-likelihood estimate under specific generative assumptions.

2.2.9 Worked examples

Example 1 (BoW from scratch — Easy). For the two-document corpus

- d_1: "*the cat sat on the mat*"
- d_2: "*the cat ate the fish*"

build the vocabulary, the document–term matrix with raw counts, and compute $\text{idf}(w)$ for each word.

Solution. $V = \{\text{the, cat, sat, on, mat, ate, fish}\}$.

	the	cat	sat	on	mat	ate	fish
d_1	2	1	1	1	1	0	0
d_2	2	1	0	0	0	1	1
df	2	2	1	1	1	1	1
idf	$\log 1 = 0$	0	$\log 2$	$\log 2$	$\log 2$	$\log 2$	$\log 2$

"*the*" and "*cat*" carry no IDF weight (they appear in every document), while *sat*, *on*, *mat*, *ate*, *fish* all carry weight $\log 2 \approx 0.69$.

Example 2 (Computing TF–IDF — Medium). A four-document corpus has $\text{df}(\textit{Tunisia}) = 1$ and $\text{df}(\textit{the}) = 4$. Document d_1 contains *Tunisia* once and *the* four times.

(a) Compute $\text{tfidf}(\textit{Tunisia}, d_1)$ and $\text{tfidf}(\textit{the}, d_1)$ using the unsmoothed formula.

(b) Comment on the result: which word's contribution dominates the document representation?

Solution. $\text{idf}(\textit{Tunisia}) = \log(4/1) = \log 4 \approx 1.39$. $\text{idf}(\textit{the}) = \log(4/4) = 0$. $\text{tfidf}(\textit{Tunisia}, d_1) = 1 \times 1.39 = 1.39$. $\text{tfidf}(\textit{the}, d_1) = 4 \times 0 = 0$. The discriminative word *Tunisia* dominates the representation, even though it appears only once. The function word *the*, which appears four times, is silenced. This is exactly the desired behaviour: a query for *Tunisia* should retrieve d_1, while a query for *the* should not.

Example 3 (Spam filtering with TF–IDF — Hard). Consider a spam-filtering corpus. The word *lottery* appears in 50 out of 5,000 training emails, all of which are spam. The word *meeting* appears in 1,200 emails, half spam, half legitimate. Argue, using TF–IDF and the discriminative power of each feature, why a logistic-regression classifier built on TF–IDF features will give a much larger weight to *lottery* than to *meeting*.

Solution. The IDF of *lottery* is $\log(5000/50) = \log 100 \approx 4.6$, while the IDF of *meeting* is $\log(5000/1200) \approx 1.4$. Already, after the IDF re-weighting, *lottery* contributes more to the representation than *meeting* per occurrence. Logistic regression then assigns weights to maximise log-likelihood; *lottery* is correlated with the spam label with probability 1, while *meeting* is correlated with the spam label with probability 0.5. The resulting weight on *lottery* reflects both the higher IDF amplification and the stronger label correlation. The classifier is therefore much more sensitive to the word *lottery* when scoring a new e-mail.

2.2.10 Python snippet: from text to TF–IDF in 5 lines

```
from sklearn.feature_extraction.text import CountVectorizer,
    TfidfVectorizer

corpus = [
```

```
    "the room was clean the room was quiet",
    "the breakfast was great the room was clean",
    "service was slow but the rooms were clean",
]

# Step 1 : raw counts
counts = CountVectorizer().fit_transform(corpus)
print("Raw counts shape:", counts.shape)
print("Raw counts dense:\n", counts.toarray())

# Step 2 : TF-IDF transform
tfidf = TfidfVectorizer().fit(corpus)
X = tfidf.transform(corpus)
print("\nVocabulary:", tfidf.get_feature_names_out())
print("TF-IDF matrix:\n", X.toarray().round(2))

# Inspect the IDF of each token
for term, score in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(f"  idf('{term}') = {score:.3f}")
```

2.2.11 Exercises

1. Build the BoW vocabulary, the document–term matrix, the IDF vector and the full TF–IDF matrix (smoothed) for the corpus

   - d_1: "*NLP is fun and useful*"
   - d_2: "*Tunisia is a country in North Africa*"
   - d_3: "*NLP is widely used in Tunisia*"

2. Show that for a corpus of N documents, the maximum possible IDF (unsmoothed) is $\log N$ and is reached when a word appears in exactly one document. Conversely, show that words appearing in every document have $\text{idf}(w) = 0$ and contribute zero TF–IDF weight.

3. Two documents, d_1 and d_2, share exactly the same set of words but with different counts. Are their BoW vectors necessarily different? Are their normalised BoW vectors (length-normalised) necessarily different?

4. TF–IDF is purely lexical and shares no information across morphological variants (*run* and *running* are unrelated columns of the document–term matrix). Propose two engineering remedies (one classical, one modern) and one situation in which each is preferable.

5. (Open-ended.) Suppose you must build a search engine over a corpus of 100,000 Tunisian-Arabic news articles. Discuss whether you would prefer a TF–IDF index, a dense-vector index (Section **??**), or a hybrid of the two. Identify a concrete query that one method handles but not the other.

2.3 Dense representations: Word2Vec, GloVe, FastText

2.3.1 Motivation

Sparse Bag-of-Words and TF–IDF representations encode lexical presence but not *meaning*. To a sparse model, the vectors of *cat* and *kitten* are orthogonal — as unrelated as *cat* and *democracy*. This severely limits generalisation: a classifier trained on documents containing *cat* cannot directly transfer its decision to documents containing *kitten*, even though the two words mean almost the same thing. *Dense word embeddings* solve this problem by mapping every word to a low-dimensional real-valued vector such that semantically similar words receive similar vectors. The idea, in a single line: *a word's meaning is determined by the company it keeps.*

2.3.2 Approach by problem: where does meaning live?

Problem. Given a corpus of one billion English sentences, how could you produce, without any human annotation, a 300-dimensional vector for each English word such that synonyms end up close together?

Solution sketch. Read the corpus and count, for every pair of words (w, c), how often they co-occur within a small context window. Words that share many contexts (*cat* / *kitten* both appear near *purr, fur, meow, pet*) will have similar count profiles. Reduce these high-dimensional count vectors to a low-dimensional latent space (e.g. via matrix factorization), and similar words will project to nearby points. The full algorithm refines this idea, but the core insight is captured by a single principle from mid-20th-century linguistics.

2.3.3 The distributional hypothesis

Definition 2.3.1 (Distributional hypothesis, Harris 1954, Firth 1957)**.** The semantic similarity of two words is well approximated by the similarity of their typical *contexts*. Formally, if $C(w)$ denotes the multiset of words appearing within a fixed-size window of w across a corpus, then

$$\mathrm{sim}_{\text{semantic}}(w_1, w_2) \approx \mathrm{sim}_{\text{distributional}}(C(w_1), C(w_2)).$$

This empirical hypothesis has held up remarkably well: every modern word-embedding algorithm is, at root, a way to operationalise it.

2.3.4 Word2Vec

Mikolov et al. (2013) introduced two shallow neural architectures that learn dense word vectors from raw text. Each word w is associated with two vectors: a *centre vector* $\mathbf{v}_w \in \mathbb{R}^d$ and a *context vector* $\mathbf{u}_w \in \mathbb{R}^d$, both stored in lookup tables and updated by gradient descent.

Skip-gram. Given a centre word, predict the surrounding context words. Formally, for a corpus tokenized as $w_1, w_2, \ldots, w_T$ and a context window of size m,

$$\mathcal{L}_{\text{skip-gram}} = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \neq 0}} \log P(w_{t+j} \mid w_t),$$

with the conditional modelled as a softmax:

$$P(c \mid w) = \frac{\exp(\mathbf{u}_c^\top \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\top \mathbf{v}_w)}.$$

CBOW (Continuous Bag-of-Words). Symmetric: predict the centre word from the average of context vectors. Faster to train but slightly less accurate on rare words.

Negative sampling. Computing the softmax over the entire vocabulary is too expensive when $|V| \sim 10^6$. The trick is to replace it with a binary classification: for each true context pair (w, c), draw k "noise" words $\tilde{c}_1, \ldots, \tilde{c}_k$ from a unigram distribution and minimise

$$-\log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) - \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{\tilde{c}_i}^\top \mathbf{v}_w).$$

This is a discriminative surrogate for the original generative loss; empirically, $k = 5$–15 is sufficient for high-quality embeddings.

2.3.5 GloVe

Pennington et al. (2014) take a complementary route: factor the global word–context co-occurrence matrix instead of training on local windows.

Setup. Let $X \in \mathbb{R}^{|V| \times |V|}$ be the matrix whose entry X_{ij} counts how often word j appears in the context of word i. The aim is to find vectors $\mathbf{v}_i, \mathbf{u}_j \in \mathbb{R}^d$ and biases b_i, c_j such that

$$\mathbf{v}_i^\top \mathbf{u}_j + b_i + c_j \approx \log X_{ij}.$$

The factorization minimises a weighted least-squares objective:

$$\mathcal{L}_{\text{GloVe}} = \sum_{i,j} f(X_{ij}) \big(\mathbf{v}_i^\top \mathbf{u}_j + b_i + c_j - \log X_{ij}\big)^2,$$

where f is a weighting function that down-weights very rare and very frequent co-occurrences.

Skip-gram and GloVe are dual. Levy and Goldberg (2014) showed that skip-gram with negative sampling implicitly factors a shifted pointwise mutual information matrix,

$$\mathbf{v}_w^\top \mathbf{u}_c \approx \log \frac{P(w, c)}{P(w)P(c)} - \log k,$$

where k is the number of negative samples. The two algorithms thus solve closely related optimisation problems with different objectives and weightings; they typically yield comparable embedding quality.

2.3.6 FastText

Bojanowski et al. (2017) extend Word2Vec by embedding *sub-word units* rather than entire words. A word is represented as the sum of vectors of its character n-grams (with $n = 3, 4, 5, 6$):

$$\mathbf{v}_{\text{word}} = \sum_{g \in \mathcal{G}_{\text{word}}} \mathbf{v}_g.$$

Two consequences:

- **No more OOV.** Any unseen word can be embedded as a sum of its n-gram vectors — including misspellings, neologisms, and morphological variants.
- **Better for morphologically rich languages.** The vectors of *run, runs, running* share most of their n-grams, so the model captures morphological similarity automatically. This is especially valuable for Arabic, Turkish, Finnish, and dialectal varieties.

2.3.7 Vector arithmetic and the analogy task

A striking empirical property of dense embeddings is that they encode semantic relations *linearly*.

Property 2.3.1 (Word analogies). For high-quality embeddings, relational analogies of the form

A is to B as C is to D

are often captured by

$$\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C \approx \mathbf{v}_D,$$

in the sense that the nearest neighbour of $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$ in the embedding space (excluding A, B, C themselves) is D.

Examples.

- Gender: $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$.
- Country–capital: $\overrightarrow{\text{Tunis}} - \overrightarrow{\text{Tunisia}} + \overrightarrow{\text{France}} \approx \overrightarrow{\text{Paris}}$.
- Verb tense: $\overrightarrow{\text{walked}} - \overrightarrow{\text{walking}} + \overrightarrow{\text{running}} \approx \overrightarrow{\text{ran}}$.

The analogy task became the dominant intrinsic-evaluation benchmark for word embeddings throughout the mid-2010s.

2.3.8 Limitations: static, monolingual, biased

- **Static.** Each word receives one vector regardless of context. The polysemous word *bank* has a single embedding that mixes its financial and riverside senses — useful as an average, useless for disambiguation. Section **??** previews the contextual fix.
- **Monolingual.** Word2Vec / GloVe / FastText vectors are trained on a single language. Cross-lingual alignment requires extra mechanisms.

- **Biases inherited from the training corpus.** The data is the model. If the corpus encodes social biases, the embeddings inherit them: $\overrightarrow{\text{nurse}}$ may end up closer to $\overrightarrow{\text{woman}}$ than to $\overrightarrow{\text{man}}$, with measurable downstream consequences. A growing body of literature studies and mitigates these effects.

2.3.9 Toward contextual embeddings

Static embeddings give one vector per word type. Modern Transformer encoders (BERT, RoBERTa, ...) instead produce one vector per word *token in context*: the same word *bank* receives different vectors when it appears in "*the bank approved my loan*" and in "*we sat on the bank of the river*". Contextualisation is the topic of Part 2 of this course; for now, treat static embeddings as the entry point and the conceptual ancestor.

2.3.10 Worked examples

Example 1 (Distributional similarity — Easy). In a small corpus, the words *cat* and *dog* both frequently co-occur with *pet*, *food*, *bowl*, while *democracy* co-occurs with *election*, *vote*, *government*. Justify intuitively why a word-embedding algorithm will place $\overrightarrow{\text{cat}}$ closer to $\overrightarrow{\text{dog}}$ than to $\overrightarrow{\text{democracy}}$.

Solution. The distributional hypothesis predicts that words with similar context distributions are placed close in embedding space. The algorithms operationalise this by training each word's vector to be predictive of (or compatible with) its observed contexts. Since *cat* and *dog* have similar context distributions, their vectors will be similar; *democracy*'s contexts are nearly disjoint, so its vector will be far from both.

Example 2 (Skip-gram softmax explicit — Medium). A toy vocabulary contains only {*cat*, *dog*, *the*}, with embeddings $\mathbf{v}_{\text{cat}} = (1, 0)$, $\mathbf{v}_{\text{dog}} = (0.9, 0.1)$, $\mathbf{v}_{\text{the}} = (0, 1)$ and context vectors $\mathbf{u}_w = \mathbf{v}_w$ for simplicity. Compute $P(dog \mid cat)$ under the skip-gram softmax.

Solution. The dot products are $\mathbf{u}_{\text{cat}}^\top \mathbf{v}_{\text{cat}} = 1$, $\mathbf{u}_{\text{dog}}^\top \mathbf{v}_{\text{cat}} = 0.9$, $\mathbf{u}_{\text{the}}^\top \mathbf{v}_{\text{cat}} = 0$.

$$P(dog \mid cat) = \frac{e^{0.9}}{e^1 + e^{0.9} + e^0} = \frac{2.46}{2.72 + 2.46 + 1} \approx \frac{2.46}{6.18} \approx 0.398.$$

The model assigns $\sim 40\%$ of its conditional mass to *dog* (close to but slightly less than *cat* itself, which would self-predict with $\sim 44\%$).

Example 3 (FastText for Tunisian Arabic — Hard). The Tunisian-Arabic verb *n7eb* ("I love") is unlikely to appear in a generic Arabic corpus, but its sub-word components (*n*, *7*, *he*, *eb*, ...) appear frequently in dialectal text. Argue why FastText would handle this word more robustly than Word2Vec, and write the formula for its vector.

Solution. Word2Vec treats *n7eb* as a single, unseen token: at inference, the lookup is undefined and the model falls back to a default `[UNK]` vector. FastText instead computes

$$\mathbf{v}_{n7eb} = \sum_{g \in \mathcal{G}(n7eb)} \mathbf{v}_g,$$

where $\mathcal{G}(n7eb)$ is the set of character n-grams of *n7eb* (with sentinels): for $n = 3$, $\mathcal{G} = \{\texttt{<n7}, \texttt{n7e}, \texttt{7eb}, \texttt{eb>}, \dots\}$. Each n-gram has been seen and embedded during training; their sum is a valid embedding even though the whole word is novel. The resulting vector

encodes morphological and orthographic similarity to other dialectal verb forms, making downstream tasks (sentiment, NER) significantly more robust.

2.3.11 Python snippet: train and explore Word2Vec

```python
# pip install gensim
from gensim.models import Word2Vec

corpus = [
    "the cat sat on the mat".split(),
    "a kitten sleeps on the couch".split(),
    "dogs and cats are common pets".split(),
    "Tunisia is a country in north africa".split(),
    "Tunis is the capital of Tunisia".split(),
    "Paris is the capital of France".split(),
]

model = Word2Vec(
    sentences=corpus, vector_size=50, window=2,
    min_count=1, sg=1, negative=5, epochs=200,
)

print("vec(cat) =", model.wv["cat"][:5], "...")
print("Most similar to 'cat':")
for w, s in model.wv.most_similar("cat", topn=3):
    print(f"  {w:<12s} cos-sim = {s:.3f}")

# Analogy: Tunis - Tunisia + France ~= Paris
analogy = model.wv.most_similar(
    positive=["France", "Tunis"], negative=["Tunisia"], topn=3,
)
print("\nTunis is to Tunisia as ? is to France:")
for w, s in analogy:
    print(f"  {w:<12s} cos-sim = {s:.3f}")
```

The toy corpus is too small to deliver high-quality vectors, but the API is identical to a corpus of billions of tokens.

2.3.12 Exercises

1. Restate the distributional hypothesis in your own words and explain why it is an *empirical* (rather than mathematical) claim. Give one sentence for which the hypothesis arguably fails.

2. Skip-gram and CBOW are dual training objectives. For each, argue qualitatively whether you expect better embeddings on rare or on frequent words, and why.

3. Show that for a uniform vocabulary distribution, the $\log P(w, c)/P(w)P(c)$ in Levy–Goldberg's analysis simplifies to $\log P(c \mid w) + \log |V|$. Comment on the consequence for the implicit objective of skip-gram.

4. Construct three analogy tests of your own — one geographic, one syntactic, and one social — and predict whether you expect a generic Word2Vec model trained on Wikipedia to solve them.

5. (Synthesis.) Discuss why FastText is a natural choice for a sentiment classifier on Tunisian-Arabic Twitter data, while Word2Vec is more suitable for a clean English Wikipedia corpus. Identify two distinct linguistic features of dialectal Arabic that drive this preference.

2.4 Semantic similarity measures

2.4.1 Motivation

Once texts have been embedded in a vector space (sparse or dense), the natural way to compare them is geometric. "How similar are these two documents?" becomes "How close are their vectors?". The choice of similarity measure is far from neutral — different measures embody different assumptions about what should count as similar. Three measures dominate practice: **cosine similarity**, **Euclidean distance**, and the **dot product**. Understanding their properties is essential for building search engines, clustering algorithms, recommender systems, and the retrieval-augmented generation pattern that sits at the heart of modern Question-Answering.

2.4.2 Approach by problem: ranking by similarity

Problem. You have embedded 10,000 news articles as TF–IDF vectors. A user submits the short query *"Tunisia football"*. Rank the articles by their relevance to the query.

Solution sketch. Embed the query with the same vectorizer used for the corpus, obtaining a query vector $\mathbf{q} \in \mathbb{R}^{|V|}$. For each article vector $\mathbf{x}_i$, compute a similarity score and sort articles in decreasing order of score. Two issues arise:

- Documents have very different lengths, hence very different ℓ_2-norms. A long article will produce a large dot product simply because its vector is large, not because it is more relevant. We need a measure invariant to magnitude.
- Articles in different topical zones should rank far apart even when they share many function words. The measure must give weight to direction, not magnitude.

Both observations point to *cosine similarity*: the cosine of the angle between query and document vectors.

2.4.3 Three classical measures

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ be two vectors.

Definition 2.4.1 (Cosine similarity)**.**

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \, \|\mathbf{b}\|_2} = \frac{\sum_{i=1}^{d} a_i b_i}{\sqrt{\sum_{i=1}^{d} a_i^2} \sqrt{\sum_{i=1}^{d} b_i^2}}.$$

The values lie in $[-1, +1]$ and equal $+1$ exactly when $\mathbf{b}$ is a positive scaling of $\mathbf{a}$.

Definition 2.4.2 (Dot product)**.**

$$\mathbf{a}^\top \mathbf{b} = \sum_{i=1}^{d} a_i b_i.$$

Faster to compute (no division or square roots) and equal to $\cos(\mathbf{a}, \mathbf{b}) \cdot \|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2$. Sensitive to the magnitudes of both vectors.

Definition 2.4.3 (Euclidean distance)**.**

$$d_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{i=1}^{d} (a_i - b_i)^2}.$$

Always non-negative; zero iff $\mathbf{a} = \mathbf{b}$. Sensitive to both magnitude and direction.

Definition 2.4.4 (Manhattan (ℓ_1) distance)**.**

$$d_1(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_1 = \sum_{i=1}^{d} |a_i - b_i|.$$

Used less often in NLP, but practical in high-dimensional spaces and when robustness to outliers matters.

2.4.4 Properties of cosine similarity

Property 2.4.1 (Magnitude invariance). For any $\lambda > 0$, $\cos(\lambda \mathbf{a}, \mathbf{b}) = \cos(\mathbf{a}, \mathbf{b})$.

Proof. Direct from the definition: scaling $\mathbf{a}$ by $\lambda > 0$ multiplies both numerator and denominator by λ, leaving the ratio unchanged. □

This single property is decisive in NLP: TF–IDF vectors of long documents are scaled-up copies of those of shorter documents on the same topic. Cosine similarity correctly considers them equivalent in topic.

Property 2.4.2 (Cosine and Euclidean for unit vectors). If $\|\mathbf{a}\|_2 = \|\mathbf{b}\|_2 = 1$, then

$$d_2(\mathbf{a}, \mathbf{b})^2 = 2 - 2\cos(\mathbf{a}, \mathbf{b}).$$

Proof. $d_2(\mathbf{a}, \mathbf{b})^2 = \|\mathbf{a}\|^2 - 2\mathbf{a}^\top \mathbf{b} + \|\mathbf{b}\|^2 = 1 - 2\mathbf{a}^\top \mathbf{b} + 1 = 2(1 - \cos(\mathbf{a}, \mathbf{b}))$ since the vectors are unit-norm. □

Remark 2.4.1. Property 2.4.2 means that, after ℓ_2-normalisation, ranking by cosine and ranking by Euclidean distance are equivalent (one is a monotonic function of the other). Practical libraries exploit this: they store ℓ_2-normalised vectors and compute dot products, which are mathematically equivalent to cosine but considerably faster.

Property 2.4.3 (Cosine similarity is not a metric). $1 - \cos(\mathbf{a}, \mathbf{b})$ does not satisfy the triangle inequality and is therefore not a metric on $\mathbb{R}^d$. (After ℓ_2-normalisation, the angular distance $\arccos(\cos(\mathbf{a}, \mathbf{b}))$ is a metric on the unit sphere.)

2.4.5 When to use which

Measure	Use when	Avoid
Cosine	Comparing texts of varying length; TF–IDF; static embeddings	Magnit
Dot product	Speed matters; vectors already ℓ_2-normalised	Magnit
Euclidean	Clustering (k-means in normalised space); coordinates with absolute meaning	Docum
Manhattan	High-dimensional vectors with outliers; robustness matters	Dense

Rule of thumb. For text classification or retrieval with TF–IDF or static embeddings, use cosine. For clustering with k-means, ℓ_2-normalise first then use Euclidean (equivalent to spherical k-means).

2.4.6 Applications

Semantic search. Embed every document; embed each incoming query; return the top-k documents by cosine similarity. This pattern powers "related articles" widgets, web search re-ranking, and the *retrieval* step of *retrieval-augmented generation* (RAG) for LLMs.

Document deduplication. In a corpus scraped from the Web, near-duplicate documents abound. Cluster cosine-similar pairs above a threshold (e.g. 0.95) and merge them.

Recommendation. Represent users and items in the same vector space; recommend items closest to the user's profile vector.

Clustering. Group documents into k topics by spherical k-means in the embedding space.

Cross-lingual similarity. If embeddings are aligned across languages, an Arabic document can be matched directly to its French paraphrase by cosine similarity, without translation.

2.4.7 The retrieval pattern

The pattern that re-appears throughout modern NLP, including in the inner loop of RAG systems, is:

$$\boxed{\text{query string} \xrightarrow[\text{embed}]{} \mathbf{q} \xrightarrow[\text{nearest neighbours}]{} \text{top-}k \text{ documents}}$$

At scale, exact nearest-neighbour search becomes infeasible. Approximate nearest-neighbour (ANN) algorithms — HNSW, FAISS, ScaNN, IVF-PQ — trade a small loss in precision for a 10–100× speed-up. We will use FAISS in the lab for a Tunisian-news search engine.

2.4.8 Worked examples

Example 1 (Cosine vs dot product on length — Easy). Two TF–IDF vectors on a vocabulary of {*Tunisia, Tunis, beach, food*} are

$$\mathbf{a} = (1, 1, 0, 0)^\top, \quad \mathbf{b} = (3, 3, 0, 0)^\top.$$

Compute the cosine similarity and the dot product. Comment.

Solution.

$$\mathbf{a}^\top \mathbf{b} = 3 + 3 = 6, \quad \|\mathbf{a}\| = \sqrt{2}, \quad \|\mathbf{b}\| = \sqrt{18} = 3\sqrt{2}.$$

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{6}{\sqrt{2} \cdot 3\sqrt{2}} = \frac{6}{6} = 1.$$

The dot product is 6, but cosine is 1 — the two vectors are scalar multiples and therefore identical in direction. Cosine correctly recognises that they encode the same topic distribution; the dot product, sensitive to magnitude, would mislead a length-sensitive ranker.

Example 2 (Ranking by similarity — Medium). A query $\mathbf{q} = (1, 1, 0, 0)^\top$ is compared against three document vectors:

$$\mathbf{d}_1 = (1, 1, 1, 0)^\top, \quad \mathbf{d}_2 = (2, 0, 0, 0)^\top, \quad \mathbf{d}_3 = (1, 1, 1, 1)^\top.$$

Rank the documents by cosine similarity to $\mathbf{q}$.

Solution. $\|\mathbf{q}\| = \sqrt{2}$, $\|\mathbf{d}_1\| = \sqrt{3}$, $\|\mathbf{d}_2\| = 2$, $\|\mathbf{d}_3\| = 2$. $\mathbf{q}^\top \mathbf{d}_1 = 2$, $\mathbf{q}^\top \mathbf{d}_2 = 2$, $\mathbf{q}^\top \mathbf{d}_3 = 2$.

$$\cos(\mathbf{q}, \mathbf{d}_1) = 2/(\sqrt{2}\sqrt{3}) \approx 0.816, \quad \cos(\mathbf{q}, \mathbf{d}_2) = 2/(\sqrt{2} \cdot 2) = 1/\sqrt{2} \approx 0.707,$$

$$\cos(\mathbf{q}, \mathbf{d}_3) = 2/(\sqrt{2} \cdot 2) \approx 0.707.$$

Ranking: $\mathbf{d}_1 \succ \mathbf{d}_2 \approx \mathbf{d}_3$. $\mathbf{d}_1$ is closest to $\mathbf{q}$ because both query terms appear and the document is short. $\mathbf{d}_2$ overweights one term; $\mathbf{d}_3$ contains both query terms but also two off-topic ones, diluting the cosine.

Example 3 (Cosine vs Euclidean for clustering — Hard). Argue why k-means clustering in TF–IDF space typically requires *prior* ℓ_2*-normalisation* of the vectors. What pathology arises if normalisation is skipped?

Solution. k-means minimises within-cluster sum of squared Euclidean distances. Without normalisation, document magnitudes dominate the distance: long documents have large $\|\mathbf{x}\|$, hence large $\|\mathbf{x} - \boldsymbol{\mu}\|$ regardless of topic. As a result, the algorithm tends to form clusters by length rather than by topic — precisely the wrong outcome.

After ℓ_2-normalisation, all vectors lie on the unit sphere; Euclidean distance becomes a monotone function of cosine similarity (Property 2.4.2), and k-means is equivalent to spherical k-means, which clusters by topic regardless of length. Practitioners therefore always normalise before k-means in NLP.

2.4.9 Python snippet: cosine, Euclidean, and FAISS

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity,
    euclidean_distances

corpus = [
    "Tunisia football national team won the cup",
    "the cup of tea I had this morning was great",
    "national football leagues across Africa",
    "great cup of coffee at this Tunis cafe",
]
```

```python

vec = TfidfVectorizer().fit(corpus)
X = vec.transform(corpus)

q = vec.transform(["Tunisia football"])

print("Cosine similarities to query:")
for i, s in enumerate(cosine_similarity(q, X).ravel()):
    print(f"  doc {i}: {s:.3f} -- {corpus[i]}")

print("\nEuclidean distances to query:")
for i, s in enumerate(euclidean_distances(q, X).ravel()):
    print(f"  doc {i}: {s:.3f} -- {corpus[i]}")

# Equivalent dot product after L2-normalisation
from sklearn.preprocessing import normalize
Xn = normalize(X, norm="l2")
qn = normalize(q, norm="l2")
print("\nDot product on normalised vectors (== cosine):")
print(np.asarray(qn @ Xn.T.toarray()).ravel().round(3))
```

For a million-document index, replace these dense methods by an approximate-nearest-neighbour engine (e.g. `faiss.IndexFlatIP` or `IndexHNSWFlat`).

2.4.10 Exercises

1. Compute, by hand, the cosine similarity, dot product, and Euclidean distance between
$$\mathbf{a} = (1, 2, 0, 1)^\top, \quad \mathbf{b} = (2, 4, 0, 2)^\top, \quad \mathbf{c} = (0, 1, 1, 1)^\top.$$
Comment on the relative ranking of $\mathbf{a}$ to $\mathbf{b}$ vs $\mathbf{a}$ to $\mathbf{c}$ under each measure.

2. Show, using only the definition, that $\cos(\mathbf{a}, \mathbf{a}) = 1$ for every non-zero vector $\mathbf{a}$, and that $\cos(\mathbf{a}, -\mathbf{a}) = -1$.

3. Prove that the angular distance $\theta(\mathbf{a}, \mathbf{b}) = \arccos(\cos(\mathbf{a}, \mathbf{b}))$ satisfies the triangle inequality on the unit sphere. (Sketch the geometric argument; full proof requires spherical geometry.)

4. Why is the dot product the preferred similarity in production search systems built on dense embeddings? (Hint: think about ℓ_2-normalisation and hardware.)

5. (Synthesis.) Sketch a Tunisian-news semantic search engine: which embedding model would you use, what similarity measure, what indexing approach (exact vs approximate), and how would you evaluate retrieval quality? Identify two failure cases that purely lexical (TF–IDF) search would miss but a dense, multilingual model would handle.

2.5 Evaluation metrics

2.5.1 Motivation

"*What is not measured cannot be improved.*" An NLP system without principled evaluation is an NLP system without progress. Different tasks demand different metrics: a classifier is judged on whether it returns the correct label, a translation system on how closely its output matches a reference, a language model on how well it predicts unseen text. This section formalises the metrics most often quoted in the literature: **Accuracy, Precision, Recall,** F_1 for classification (Section **??**), and **Perplexity, BLEU, ROUGE** for generation (Section **??**). We pay equal attention to their pitfalls — a single number is never the whole story.

2.5.2 Approach by problem: the misleading 95% accuracy

Problem. You build a classifier that detects whether a tweet contains hate speech. Hate-speech tweets account for 5% of your corpus. A trivial baseline that always predicts *not hate speech* achieves 95% accuracy. Is this baseline useful? What metric reveals its uselessness?

Solution sketch. The baseline misses every actual case of hate speech — its *recall on the positive class* is 0. Accuracy is misleading whenever classes are imbalanced: it is dominated by the easy majority class. Reporting *precision and recall on the positive class*, or the F_1 score, makes the baseline's failure visible immediately. This single example motivates everything that follows.

2.5.3 Classification metrics

The confusion matrix

Definition 2.5.1 (Confusion matrix, binary case)**.** For a binary classifier with output $\hat{y} \in \{0, 1\}$ and ground-truth label $y \in \{0, 1\}$, define four counts on a test set of size N:

	Predicted positive	Predicted negative
Actual positive	TP (true positive)	FN (false negative)
Actual negative	FP (false positive)	TN (true negative)

By construction, $\mathrm{TP} + \mathrm{FP} + \mathrm{FN} + \mathrm{TN} = N$.

Accuracy

Definition 2.5.2 (Accuracy)**.**

$$\text{Accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}.$$

The fraction of predictions that match the ground truth.

Accuracy is intuitive but misleading under class imbalance. Use it only when the class distribution is roughly balanced and the costs of false positives and false negatives are comparable.

Precision and Recall

Definition 2.5.3 (Precision and Recall)**.**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Library analogy. Imagine searching a library for books on machine learning.

- **Precision**: of the books you brought home, how many are about ML?
- **Recall**: of all ML books in the library, how many did you bring home?

A search that returns only the most obvious ML book has high precision but low recall. A search that returns every book in the library has perfect recall but very low precision.

When to optimise which.

- **Precision-critical**: spam detection (a false positive blocks a legitimate email — much worse than letting one spam through).
- **Recall-critical**: medical screening (missing a tumour is far worse than a false alarm — patients can be re-tested).
- **Balanced**: most generic NLP tasks (NER, sentiment, topic classification).

F-measure

Definition 2.5.4 (F_1 score)**.** The F_1 score is the harmonic mean of Precision and Recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The harmonic mean penalises imbalance more harshly than the arithmetic mean: F_1 is high only when *both* Precision and Recall are high.

Property 2.5.1 (Bounds of F_1). $0 \leq F_1 \leq 1$. $F_1 = 1$ iff Precision = Recall = 1. If either is zero, $F_1 = 0$.

Remark 2.5.1 (F_β generalisation). The general F_β score weights recall β times more than precision:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}.$$

F_2 favours recall (medical), $F_{0.5}$ favours precision (anti-spam).

Multi-class extensions

For K-class problems, compute Precision/Recall/F_1 per class, then aggregate:

- **Macro-average**: simple average of the K per-class metrics. Treats all classes equally.
- **Micro-average**: pool all decisions across classes, then compute the metric. Dominated by frequent classes.
- **Weighted average**: weight each class by its prevalence.

2.5.4 Generation metrics

Perplexity

Generation models do not produce a discrete label; they produce a probability distribution over the next token. *Perplexity* measures how well the model predicts a held-out test sequence.

Definition 2.5.5 (Perplexity)**.** For a language model P_θ and a sequence $w_1, \ldots, w_N$ from a held-out set, the perplexity is

$$\mathrm{PPL}(P_\theta) = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P_\theta(w_i | w_1,\ldots,w_{i-1})} = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N} \log P_\theta(w_i \mid w_1, \ldots, w_{i-1})\Big).$$

Intuition. If perplexity is k, the model is on average "hesitating between k equally likely words" at each position. Lower is better; perplexity of 1 means perfect prediction.

Property 2.5.2 (Lower bound). $\mathrm{PPL}(P_\theta) \geq 1$, with equality iff P_θ assigns probability 1 to every observed token.

Remark 2.5.2 (A caveat). Low perplexity does not guarantee high-quality output. A model can have low perplexity on test text yet produce repetitive or incoherent generation. Perplexity is therefore a useful but partial proxy.

BLEU (translation)

Definition 2.5.6 (BLEU score, simplified)**.** The Bilingual Evaluation Understudy score (Papineni et al., 2002) compares a candidate translation c to one or several reference translations $\{r_1, \ldots, r_M\}$. It combines a *modified n-gram precision* p_n for $n = 1, \ldots, N$ (typically $N = 4$) and a *brevity penalty* BP:

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{N} \frac{1}{N} \log p_n\right),$$

where

$$\mathrm{BP} = \begin{cases} 1 & \text{if } |c| > |r| \\ \exp(1 - |r|/|c|) & \text{otherwise} \end{cases}$$

penalises candidates shorter than the reference.

Modified n-gram precision. For each n-gram in the candidate, count its occurrences in the candidate (call it count_c) and clip it by the maximum occurrences across the references. Sum the clipped counts and divide by the total candidate n-grams.

Range. $0 \leq \mathrm{BLEU} \leq 1$ (often reported as a percentage). Modern strong systems on news translation reach 30–50 BLEU; human translation typically scores in the same range, illustrating that BLEU is a useful but imperfect proxy for human judgement.

ROUGE (summarization)

Definition 2.5.7 (ROUGE-N, simplified)**.** For a candidate summary c and a reference r,

$$\text{ROUGE-N} = \frac{|\, n\text{-grams}_c \cap n\text{-grams}_r \,|}{|\, n\text{-grams}_r \,|} = \text{Recall}_n.$$

ROUGE-1 uses unigrams; ROUGE-2 uses bigrams.

Definition 2.5.8 (ROUGE-L)**.** ROUGE-L uses the length of the longest common subsequence (LCS) between c and r:

$$\text{ROUGE-L} = \frac{\text{LCS}(c, r)}{\max(|c|, |r|)}.$$

BLEU vs ROUGE. BLEU is precision-oriented ("what fraction of the candidate is correct?"); ROUGE is recall-oriented ("what fraction of the reference is captured?"). For *translation* we typically want both, but with a strong precision bias — hence BLEU. For *summarisation* we want to avoid missing important content, so recall is paramount — hence ROUGE.

2.5.5 Summary table

Metric	Task	Captures	Range
Accuracy	Classification	Overall correctness	$[0, 1]$
Precision	Classification	Quality of positive predictions	$[0, 1]$
Recall	Classification	Coverage of positive cases	$[0, 1]$
F_1	Classification	Harmonic balance of P and R	$[0, 1]$
Perplexity	Language modelling	Predictive quality (geometric mean)	$[1, \infty)$
BLEU	Translation	n-gram precision with brevity penalty	$[0, 1]$
ROUGE-N	Summarisation	n-gram recall vs reference	$[0, 1]$
ROUGE-L	Summarisation	Longest common subsequence recall	$[0, 1]$

Remark 2.5.3 (Modern complementary metrics). Each of the above is purely surface-form: it counts n-gram or token matches and ignores meaning. Modern alternatives such as *BERTScore* (semantic similarity via contextual embeddings), *LLM-as-a-judge* (a strong language model rates the candidate), *RAGAS* (evaluation suite for retrieval-augmented generation) and *MT-Bench* (multi-turn instruction-following) attempt to measure semantic and pragmatic adequacy. We encounter them in later parts of the course.

2.5.6 Worked examples

Example 1 (Confusion matrix — Easy). A spam detector tested on 1000 emails (300 spam, 700 legitimate) produces 250 true positives, 50 false negatives, 40 false positives, 660 true negatives. Compute Accuracy, Precision, Recall, and F_1 for the spam class.

Solution.

$$\begin{aligned}
\text{Accuracy} &= (250 + 660)/1000 = 0.91, \\
\text{Precision} &= 250/(250 + 40) \approx 0.862, \\
\text{Recall} &= 250/(250 + 50) \approx 0.833, \\
F_1 &= \frac{2 \cdot 0.862 \cdot 0.833}{0.862 + 0.833} \approx 0.847.
\end{aligned}$$

Example 2 (Imbalanced classes — Medium). On a corpus where 99% of tweets are non-toxic, a classifier predicts *non-toxic* for every input. Compute its accuracy, precision, recall, and F_1 on the toxic class.

Solution. TP $= 0$, FN $= 1\%$ of corpus, FP $= 0$, TN $= 99\%$ of corpus. Accuracy $= 0.99$. Precision $= 0/0$ (undefined; conventionally 0). Recall $= 0/1\% = 0$. $F_1 = 0$. The accuracy of 99% celebrates a model that detects no toxicity at all. F_1 correctly reveals it as useless on the task we care about. *Always inspect class-conditional metrics under imbalance.*

Example 3 (BLEU vs ROUGE on summary — Hard). Reference: "*Tunisia won the African Cup of Nations.*" (8 tokens) Candidate: "*Tunisia won the cup.*" (4 tokens)

(a) Compute the unigram precision of the candidate (BLEU-1, no brevity penalty for now). (b) Compute ROUGE-1. (c) Compute the brevity penalty and the full BLEU-1.

Solution. (a) Candidate unigrams: {Tunisia, won, the, cup}. All four appear in the reference. Modified precision $= 4/4 = 1.0$.

(b) ROUGE-1 unigram recall: reference unigrams (after lowercasing) {tunisia, won, the, african, cup, of, nations}, 7 distinct. Candidate covers {tunisia, won, the, cup}, 4 tokens. ROUGE-1 $= 4/7 \approx 0.571$.

(c) Brevity penalty: $|c| = 4 < |r| = 8$, so BP $= \exp(1 - 8/4) = \exp(-1) \approx 0.368$. BLEU-1 $= 0.368 \times 1.0 \approx 0.368$.

The candidate has high precision (everything it says is correct) but low recall (it omits half the reference). BLEU's brevity penalty captures part of this issue; ROUGE captures it directly through recall.

2.5.7 Python snippet: classification + generation metrics

```python
# Classification
from sklearn.metrics import precision_recall_fscore_support,
    classification_report

y_true = [1, 1, 1, 0, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 1, 0, 1, 0]

p, r, f1, _ = precision_recall_fscore_support(
     y_true, y_pred, pos_label=1, average="binary",
)
print(f"Precision = {p:.3f}, Recall = {r:.3f}, F1 = {f1:.3f}")
print("\nFull report:")
print(classification_report(y_true, y_pred, target_names=["neg", "
    pos"]))

# BLEU and ROUGE
# pip install nltk rouge-score
from nltk.translate.bleu_score import sentence_bleu,
    SmoothingFunction
from rouge_score import rouge_scorer

reference = "Tunisia won the African Cup of Nations".split()
candidate = "Tunisia won the cup".split()

```

```
bleu = sentence_bleu(
    [reference], candidate,
    weights=(1.0, 0, 0, 0),  # BLEU-1
    smoothing_function=SmoothingFunction().method1,
)
print(f"\nBLEU-1 = {bleu:.3f}")

scorer = rouge_scorer.RougeScorer(["rouge1", "rougeL"], use_stemmer=True)
scores = scorer.score(" ".join(reference), " ".join(candidate))
for k, v in scores.items():
    print(f"  {k}: P={v.precision:.3f} R={v.recall:.3f} F={v.fmeasure:.3f}")
```

2.5.8 Exercises

1. For a NER tagger that finds 80 true entities, misses 20, and produces 30 spurious entities, compute Precision, Recall, and F_1 on the entity class.

2. Show that for any binary classifier $F_1 \le \min(\text{Precision}, \text{Recall})$, with equality iff Precision $=$ Recall.

3. A language model assigns probabilities $0.5, 0.5, 0.25$ to the three tokens of a held-out sequence. Compute its perplexity. What perplexity would a uniform model over a vocabulary of $|V| = 10{,}000$ achieve?

4. For the reference "*The cat is on the mat*" and the candidate "*The cat sits on the mat*", compute BLEU-2 and ROUGE-2 by hand. What linguistic phenomenon is being measured (or missed) by each?

5. (Synthesis.) Design an evaluation protocol for a Tunisian-Arabic dialect-to-French translation system. Identify which metrics you would report, which biases each metric carries (e.g. BLEU's surface-form sensitivity), and how you would complement them with human evaluation.